

Progettazione di Applicazioni Mobili

Flutter Programming

Dart

Stefano Burigat

Dipartimento di Scienze Matematiche, Informatiche e
Fisiche - Università di Udine

stefano.burigat@uniud.it



A basic Dart program

```
// Define a function.  
printInteger(int aNumber) {  
    print("The number is " + aNumber.toString()); // Print to console.  
}  
  
// This is where the app starts executing.  
main() {  
    var number = 42; // Declare and initialize a variable.  
    printInteger(number); // Call a function.  
}
```

- Everything you can place in a variable is an object, and every object is an instance of a class. Even numbers, functions, and null are objects. All objects inherit from the Object class.

Comments

- Dart supports single-line comments using the familiar `//` character sequence. The compiler ignores anything following this on the line

```
// Define the user's age.  
int age = 25; // The age is 25
```

- A second form is multi-line comments using the `/*` and `*/` marker sequences. Anything between those two sequences is ignored

```
/*  
  This function calculates account balance  
  using the Miller-Hawthorne method  
  of value calculation.  
*/
```

Variables

- Dart is a strongly typed language, supporting common types such as *int*, *double*, *String*, *bool* as well as collection types such as *List*, *Set*, and *Map*
- You can declare and initialize a variable in two ways:

```
var x = "Mel Brooks";  
String x = "Mel Brooks";
```

When you do *var x*, Dart will infer the type from the value assigned

- There is also a dynamic type annotation that tells Dart that what a variable references can change in type over time:

```
dynamic x = "Mel Books";
```

Null safety

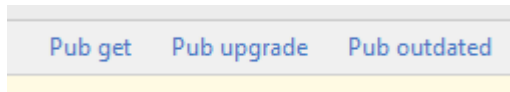
- Since version 2.12, Dart comes with sound null safety
- With sound null safety, variables can never be assigned the value *null*
- Since *null* is the default value of a variable that is not initialized, this means that you always need to initialize variables before they are used
- You can specify that a type of a variable is nullable by adding a ? after the type (e.g. *int? i*). Then, the variable can contain either a null or a value of the defined type.

Variable visibility

- In Java and many other OOP languages, you typically need to specify the visibility of a variable using keywords like `public`, `private`, and `protected`
- Dart is different: everything (including variables, functions, classes) is public unless it begins with an underscore, which marks it as being private to its library
 - Note that each file in a Dart project can be considered its own library (unless it is explicitly annotated as being part of a specific library) so marking something as private with the underscore makes it unusable outside of the file

Libraries

- Libraries are used in Dart to make code modular and shareable: a library provides some external API to other code that uses it
- To get Dart packages, you use the **pub** package manager
 - You can find publicly available packages on the pub.dev site
 - The pubspec.yaml file is used to declare dependencies in your project
 - Opening the pubspec file in Android Studio reveals the interface you can use to use the pub package manager



Libraries

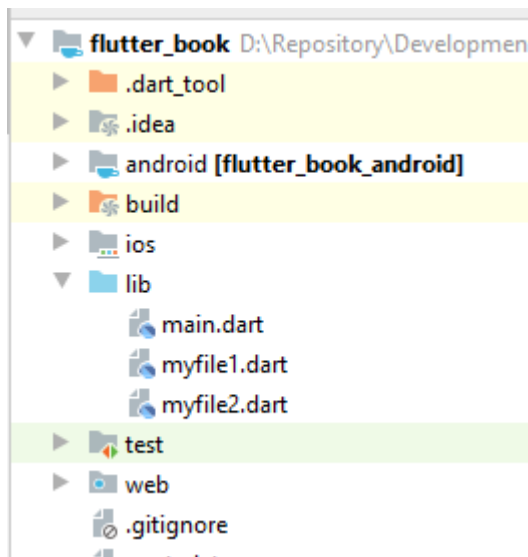
- To use a library, you need to use the *import* keyword and the URI of the library (typically with *dart:* or *package:* schemas for platform or external libraries, respectively)

```
import "dart:html";  
import "package:someLib.dart";
```

- To use external libraries you have first to specify such libraries in the pubspec file

Libraries

- If your project consists of multiple files and you need to access functions/classes/variables across files you need to use the import keyword with the local path of the file to use



A screenshot of the 'main.dart' file in an IDE. The file is open in a tab alongside 'myfile1.dart' and 'myfile2.dart'. The code is as follows:

```
1 import 'myfile2.dart';
2
3 void main() {
4   print("Testing...");
5   myPrint();
6   print(counter);
7 }
```

A screenshot of the 'myfile2.dart' file in an IDE. The file is open in a tab alongside 'main.dart' and 'myfile1.dart'. The code is as follows:

```
1 void myPrint() {
2   print("MyClass testing...");
3   print(counter);
4 }
5 int counter = 0;
```

Constants and Final Values

- *const* and *final* define a variable as being an immutable value
- *final* means you can only set it once, but you can do so at runtime, while *const* means you can only set it once, but its value must be knowable at compile-time

final x = DateTime.now(); // works

const x = DateTime.now(); // does not work

- They both work with type annotations as well

Data types

- Dart offers a String type, which is a sequence of UTF-16 code units
- Strings can be initialized using either single or double quotes
- Strings can include expressions using the `${expression}` syntax. If the expression refers to an identifier, then you can drop the curly braces

```
String s1 = "Rickety Rocket";  
String s2 = "${s1} blast off!";  
String s3 = '$s1 blast off!';  
print (s2);  
print (s3);
```

Data types

- Your typical integer numeric values have a type of `int`
- A double precision floating point number, as specified by the IEEE 754 standard, has a type of `double`
- A numeric can be turned into a string using the `toString()` method of the `int` and `double` classes and a string can be turned into a number with the `parse()` method of the `int` and `double` classes

```
int i = 5;  
double d = 5.5;  
String si = i.toString();  
String sd = d.toString();
```

```
String si = "5";  
String sd = "5.5";  
int i = Integer.parseInt(si);  
double d = Double.parseDouble(sd);
```

Data types

- Boolean values are of type `bool`, and only two objects have boolean values: the keywords `true` and `false`
- The `List` class in Dart is akin to an array in most languages. An instance of one is a list of values which are defined with syntax identical to JavaScript

```
List lst = [ 1, 2, 3 ];
```

- A list uses a zero-based indexing scheme, allows you to access elements by index, and has several methods available to add, remove, sort, etc.

```
print (lst[1]);  
lst.add(4);  
lst.removeLast();  
print(lst.indexOf(4));
```

Data types

- Dart offers a Set class, which is similar to List, but it's an unordered list, which means you can't retrieve elements by index, you have to use methods `contains()` and `containsAll()` instead

```
Set cookies = Set();
cookies.addAll([ "oatmeal", "chocolate", "rainbow" ]);
cookies.add("oatmeal"); // No harm, no foul
cookies.remove("chocolate");
print(cookies);
print(cookies.contains("oatmeal"));
print(cookies.containsAll([ "chocolate", "rainbow" ]));
```

Data types

- Dart also has a Map class, sometimes called a dictionary or a hash, an instance of which can be considered as a set of key/value pairs and can be created in different ways

```
var actors = {  
  "Ryan Reynolds" : "Deadpool",  
  "Hugh Jackman" : "Wolverine"  
};  
print(actors);
```

```
var actresses = Map();  
actresses["scarlett johansson"] = "Black Widow";  
actresses["Zoe Saldana"] = "Gamora";  
print (actresses);
```

```
var movies = Map<String, int>();  
movies["Iron Man"] = 3;  
movies["Thor"] = 3;  
print(movies);
```

```
print(actors["Ryan Reynolds"]);  
print(actresses["Elizabeth Olsen"]);  
movies.remove("Thor");  
print(movies);  
print(actors.keys);  
print(actresses.values);
```

Functions

- In Dart, functions are objects (of type `Function`)
 - This means that functions can be assigned to variables and can be passed as parameters
- There's one key stand-alone function you're already aware of, *main()*, which is the entry point to an application
- Dart supports arrow, or lambda, notation for defining functions (only an expression can be put after the arrow, not a statement)

```
talk1() { print("abc"); }  
talk2() => print("abc");
```

Functions

- A function can have *required positional* parameters, followed by either *named* parameters or *optional positional* parameters (but not both)
- Flutter widget constructors use only named parameters, even for parameters that are required (this is obtained by using the *required* keyword before the parameter)

Functions

- Wrapping a set of function parameters in [] marks them as optional positional parameters

```
String say(String from, String msg, [String? device]) {  
    var result = '$from says $msg';  
    if (device != null) {  
        result = '$result with a $device';  
    }  
    return result;  
}
```

```
assert(say('Bob', 'Howdy') == 'Bob says Howdy');
```

```
assert(say('Bob', 'Howdy', 'smoke signal') ==  
    'Bob says Howdy with a smoke signal');
```

Functions

- Wrapping a set of function parameters in `{}` marks them as named parameters

```
/// Sets the [bold] and [hidden] flags ...  
void enableFlags({bool? bold, bool? hidden}) {...}
```

- When calling the function, you must specify named parameters using *paramName: value*

```
enableFlags(bold: true, hidden: false);
```

- Named parameters are optional unless they're explicitly marked as *required*

Functions

- You can define default values for both named and optional positional parameters
 - If no default value is provided, the default value is null

```
/// Sets the [bold] and [hidden] flags ...  
void enableFlags({bool bold = false, bool hidden = false}) {...}  
  
// bold will be true; hidden will be false.  
enableFlags(bold: true);
```

```
String say(String from, String msg, [String device = 'carrier pigeon']) {  
  var result = '$from says $msg with a $device';  
  return result;  
}  
  
assert(say('Bob', 'Howdy') == 'Bob says Howdy with a carrier pigeon');
```

Functions

- You can pass a function as a parameter to another function

```
void printElement(int element) {  
    print(element);  
}  
  
var list = [1, 2, 3];  
  
// Pass printElement as a parameter.  
list.forEach(printElement);
```

- You can also assign a function to a variable

```
var loudify = (msg) => '!!! ${msg.toUpperCase()} !!!';  
assert(loudify('hello') == '!!! HELLO !!!');
```

Functions

- While most functions have a name, they don't have to, they can also be anonymous

```
main() {  
    var bands = [ "Dream Theater", "Kamelot", "Periphery" ];  
    bands.forEach((band) {  
        print("${bands.indexOf(band)}: $band");  
    });  
}
```

Here, there's a function passed to the `forEach()` method of the List object `bands`, but it has no name and as a result, it only exists for the lifetime of the execution of `forEach()`

void

- In Dart, if a function doesn't explicitly return anything, then you can omit a return type entirely; you don't even need to put *void* in front of it like most languages (although you are free to do so if you prefer)
- If you do put *void* in front of a function though, you will then get a compile-time error if you try to return anything (with the exception of *null*)

Flow control

- Looping in Dart takes on the familiar for, do, and while loop forms

```
for (var i = 0; i < 10; i++) {  
  print(i);  
}
```

- There are also a for-in and forEach form, if the target class is iterable

```
List starfleet = [ "1701", "1234", "1017", "2610", "7410" ];  
main() {  
  for (var shipNum in starfleet) {  
    print("NCC-" + shipNum);  
  }  
}  
  
main() {  
  starfleet.forEach((shipNum) => print("NCC-" + shipNum));  
}
```

Flow control

- The do and while loops offer the typical two forms, do-while and while-do

```
while (!isDone()) {  
    // Do something  
}  
  
do {  
    showStatus();  
} while (!processDone());
```

- Dart also offers a switch structure (on String and int types)

```
switch (someVariable) {  
    case 1:  
        // Do something  
        break;  
    case 2:  
        // Do something else  
        break;  
    default:  
        // It wasn't 1 or 2  
        break;  
}
```

Flow control

- The common if statement is also present in Dart

```
if (mercury == true || venus == true ||
    earth == true || mars == true
) {
  print ("It's an inner planet");
} else if (jupiter || saturn || uranus || neptune) {
  print ("It's an outer planet");
} else {
  print("Poor Pluto, you are NOT a planet");
}
```

- Note that in Dart, condition expressions must always evaluate to a boolean value, nothing else is allowed

Operators

Description	Operator
unary postfix	<i>expr</i> ++ <i>expr</i> -- () [] . ?.
unary prefix	- <i>expr</i> ! <i>expr</i> ~ <i>expr</i> ++ <i>expr</i> -- <i>expr</i> await <i>expr</i>
multiplicative	* / % ~/
additive	+ -
shift	<< >> >>>
bitwise AND	&
bitwise XOR	^
bitwise OR	
relational and type test	>= > <= < as is is!
equality	== !=
logical AND	&&
logical OR	
if null	??
conditional	<i>expr1</i> ? <i>expr2</i> : <i>expr3</i>
cascade	.. ?..
assignment	= *= /= += -= &= ^= etc.

Object orientation: classes

- Dart is object-oriented, which means we're dealing with classes and objects
- Defining a class is as simple as using the *class* keyword followed by the name of the class

```
class Hero { }
```

- All class declarations in Dart are top-level since Dart does not support nested classes

Object orientation: instance variables

- Classes frequently have instance variables like so

```
class Hero {  
    String? firstName;  
    String? lastName;  
    int age = 0;  
}
```

- Any instance variable that you don't initialize with a value begins with a value of null
- Instance variables can be marked as *static*, which means you can use them without instantiating the class

```
class MyClass {  
    static String greeting = "Hi";  
}  
  
main() {  
    print(MyClass.greeting);  
}
```

Object orientation: methods

- Classes can also have member functions, called methods

```
class Hero {  
    String? firstName;  
    String? lastName;  
    String sayName(){  
        return "$lastName, $firstName";  
    }  
}
```

```
main() {  
    Hero h = new Hero ();  
    h.firstName = "Luke";  
    h.lastName = "Skywalker";  
    print(h.sayName());  
}
```

The new keyword is optional

- Methods can be marked as *static*, just like instance variables

```
class MyClass {  
    static sayHi() {  
        print("Hi");  
    }  
}  
  
main() {  
    MyClass.sayHi();  
}
```

Object orientation: constructors

- Classes frequently have constructors, special functions that execute when an instance of them is created. The constructor always has the same name as the class and doesn't have a return type

```
class Hero {  
    String? firstName;  
    String? lastName;  
    Hero(String fn, String ln){  
        firstName = fn;  
        lastName = ln;  
    }  
    String sayName(){  
        return "$lastName, $firstName";  
    }  
}
```

```
main() {  
    Hero h = new Hero("Luke", "Skywalker");  
    print(h.sayName());  
}
```

Object orientation: constructors

- Because a constructor that just sets instance variable values is such a common pattern, Dart has a shortcut constructor form for this

```
class Hero {  
  String? firstName;  
  String? lastName;  
  Hero(this.firstName, this.lastName);  
  String sayName(){  
    return "$lastName, $firstName";  
  }  
}
```

- The *this* keyword references the current instance of the class a block of code is executing within. Typically, you should only use *this* when there is a naming conflict

Object orientation: subclasses

- Subclasses can be defined by using the extends keyword

```
class Hero {  
    String? firstName;  
    String? lastName;  
    Hero.build(this.firstName, this.lastName);  
    String sayName(){  
        return "$lastName, $firstName";  
    }  
}  
  
class UltimateHero extends Hero {  
    UltimateHero(fn, ln) : super.build(fn, ln);  
    String sayName(){  
        return "Jedi $lastName, $firstName";  
    }  
}
```

The Hero.build() method is a named constructor. Named constructor can be used to implement multiple constructors for a class or to provide extra clarity.

Subclasses don't inherit constructors from their superclass.

The *super* keyword allows you to call methods or access variables in the parent class (super.build calls the Hero class constructor)

Object orientation: interfaces

- Dart doesn't distinguish the notion of classes and interfaces like most other object-oriented languages do
- A Dart class also implicitly defines an interface

```
class UltimateHero implements Hero {  
  @override  
  String? firstName;  
  @override  
  String? lastName;  
  UltimateHero(this.firstName, this.lastName);  
  String sayName(){  
    return "Jedi $lastName, $firstName";  
  }  
}
```

The `@override` is a metadata annotation. It's necessary to indicate to Dart that we are overriding the superclass's getter and setter for the two marked fields

Generics

- Generics are used to inform Dart about the type of something
- If you write

```
var ls = List<String>();
```

...then Dart knows that the list `ls` can only contain Strings. It will enforce that type safety at compile-time

- Lists (and Maps) can be defined generically as shown earlier, and you can use the literal form as well:

```
var brands = <String>[ "Ford", "Pepsi", "Disney" ];
```

async/await

- Dart libraries are full of functions that return Future or Stream objects
 - These functions are asynchronous: they return after setting up a possibly time-consuming operation (such as I/O), without waiting for that operation to complete
- The `async` and `await` keywords support asynchronous programming, letting you write asynchronous code that looks similar to synchronous code

async/await

- Here's some code that uses await to wait for the result of an asynchronous function:

```
await lookUpVersion();
```

- To use await, code must be in an async function:

```
Future checkVersion() async {  
  var version = await lookUpVersion();  
  // Do something with version  
}
```

Although an async function might perform time-consuming operations, it doesn't wait for those operations. Instead, the async function executes only until it encounters its first await expression. Then it returns a Future object, resuming execution only after the await expression completes.

Future

- A future (lower case “f”) is an instance of the Future (capitalized “F”) class and represents the result of an asynchronous operation
- A Future<T> instance produces a value of type T
- If a future doesn’t produce a usable value, then the future’s type is Future<void>
- A future can be in one of two states: uncompleted or completed
 - When you call an asynchronous function, it returns an uncompleted future. That future is waiting for the function’s asynchronous operation to finish
 - If the asynchronous operation succeeds, the future completes with a value. Otherwise it completes with an error

async/await example

```
void createOrderMessage() {  
    var order = fetchUserOrder();  
    print('Your order is: $order');  
}  
  
Future<String> fetchUserOrder() =>  
    // Imagine that this function is  
    // more complex and slow.  
    Future.delayed(  
        Duration(seconds: 2),  
        () => 'Large Latte',  
    );  
  
void main() {  
    print('Fetching user order...');  
    createOrderMessage();  
    print("Order fetched ?");  
}
```

```
Future createOrderMessage() async {  
    var order = await fetchUserOrder();  
    print('Your order is: $order');  
}  
  
Future<String> fetchUserOrder() =>  
    // Imagine that this function is  
    // more complex and slow.  
    Future.delayed(  
        Duration(seconds: 2),  
        () => 'Large Latte',  
    );  
  
void main() {  
    print('Fetching user order...');  
    createOrderMessage();  
    print("Still fetching...");  
}
```

async/await example

```
Future<String> createOrderMessage() async {  
  var order = await fetchUserOrder();  
  return 'Your order is: $order';  
}  
  
Future<String> fetchUserOrder() =>  
  // Imagine that this function is  
  // more complex and slow.  
  Future.delayed(  
    Duration(seconds: 2),  
    () => 'Large Latte',  
  );  
  
Future<void> main() async {  
  print('Fetching user order...');  
  print(await createOrderMessage());  
  print("User order received and printed...");  
}
```